Java 技术栈
面试八股文大全

Java · JVM · Spring Boot · 微服务 · AOP · 事务

涵盖 50+ 高频面试题 · 深度解析 · 代码示例

适用于中高级 Java 开发工程师面试准备

目 录

一、Java 基础与核心知识 3
1.1 Java 语言基础 3
1.2 集合框架 4
1.3 多线程与并发 5
二、JVM（Java 虚拟机） 8
2.1 内存区域 8
2.2 垃圾回收（GC） 8
2.3 类加载机制 9
2.4 JVM 调优 10
三、Spring Boot 框架 11
3.1 核心原理 11
3.2 Spring IoC 与 Bean 生命周期 11
3.3 Spring MVC 12
四、微服务架构 13
4.1 微服务基础 13
4.2 服务注册与发现 13
4.3 熔断降级与限流 13
4.4 分布式相关 14
五、AOP（面向切面编程） 16
5.1 AOP 基础 16
5.2 AOP 实现原理 16
5.3 自定义 AOP 注解实战 17
六、事务 19
6.1 事务基础 19
6.2 Spring 事务管理 19
6.3 事务失效场景（高频必问） 20
6.4 分布式事务 22
七、补充高频题 23
7.1 设计模式在 Spring 中的应用 23
7.2 MySQL 锁机制 23

一、Java 基础与核心知识

1.1 Java 语言基础

Q：== 和 equals() 的区别是什么？

`==` 对于基本数据类型比较的是值，对于引用类型比较的是内存地址（即是否为同一个对象）。

`equals()` 是 Object 类的方法，默认实现也是比较内存地址，但很多类（如 String、Integer）重写了该方法，使其比较的是对象的内容。

重点： 重写 `equals()` 时必须同时重写 `hashCode()`，否则在使用 HashMap、HashSet 等基于哈希的集合时会出现逻辑错误。

```
// 示例：== 与 equals 的区别
String s1 = new String("hello");
String s2 = new String("hello");
System.out.println(s1 == s2);      // false（不同对象）
System.out.println(s1.equals(s2)); // true（内容相同）


// Integer 缓存池陷阱
Integer a = 127, b = 127;
System.out.println(a == b);  // true（缓存范围 -128~127）
Integer c = 128, d = 128;
System.out.println(c == d);  // false（超出缓存范围）
```

Q：String、StringBuilder 和 StringBuffer 的区别？

String 是不可变类（immutable），底层使用 `final char[]`（JDK 9 后改为 `final byte[]`），每次修改都会创建新对象。StringBuilder 和 StringBuffer 都是可变的，区别在于 StringBuffer 的方法使用了 `synchronized` 关键字，是线程安全的但性能较低；StringBuilder 非线程安全但性能更高。

特性	String	StringBuilder	StringBuffer
可变性	不可变	可变	可变
线程安全	安全（不可变）	不安全	安全（synchronized）
性能	拼接慢	最快	较快
适用场景	少量字符串操作	单线程大量拼接	多线程共享拼接

Q：final、finally、finalize 的区别？

`final` 是修饰符：修饰类表示不可继承，修饰方法表示不可重写，修饰变量表示不可重新赋值（但引

用类型的对象内容仍可修改）。`finally` 是异常处理的一部分，无论是否发生异常都会执行（除非 JVM 退出或线程被中断）。`finalize()` 是 Object 类的方法，在对象被 GC 回收前调用，但不推荐使用，Java 9 已将其标记为 @Deprecated。

Q：接口和抽象类的区别？

特性	抽象类	接口
关键字	abstract class	interface
继承/实现	单继承	多实现
构造方法	可以有	不能有
成员变量	任意类型	只能 public static final
方法	抽象方法 + 具体方法	Java 8 起支持 default/static 方法
设计理念	is-a 关系	has-a / can-do 能力约定

1.2 集合框架

Q：HashMap 的底层实现原理？（高频）

JDK 1.8 中 HashMap 底层采用**「数组 + 链表 + 红黑树」**的结构。数组是 `Node<K,V>[]`，默认初始容量为 16，负载因子为 0.75。

put 操作流程：对 key 的 hashCode 进行扰动处理（高 16 位异或低 16 位）→ 通过 `(n-1) & hash` 计算数组下标 → 如果发生哈希冲突，以链表形式存储 → 当链表长度超过 8 且数组长度 >= 64 时，链表转化为红黑树（O(n) → O(log n)）→ 当红黑树节点数 < 6 时退化为链表。

```
// HashMap 扰动函数源码（JDK 1.8）
static final int hash(Object key) {
    int h;
    // 高 16 位异或低 16 位，让高位也参与运算，减少碰撞
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}


// 定位桶的位置（等价于 hash % n，但位运算更快）
int index = (n - 1) & hash;
```

> 💡 **面试加分点**
> 扩容时容量翻倍，JDK 1.8 优化了 rehash：只需判断新增的一个 bit 是 0 还是 1，决定元素在原位置还是「原位置 + 旧容量」的位置，避免了重新计算 hash。

Q：ConcurrentHashMap 的实现原理？JDK 1.7 和 1.8 的区别？

特性	JDK 1.7	JDK 1.8
数据结构	Segment[] + HashEntry[]	Node[] + 链表/红黑树
锁机制	Segment 分段锁（ReentrantLock）	CAS + synchronized（锁桶头节点）
锁粒度	Segment（默认 16 段）	单个桶（Node）
并发度	最多 16 线程并发写	理论上等于桶数量
size() 实现	多次不加锁尝试 → 全部加锁	baseCount + CounterCell（类似 LongAdder）

Q：ArrayList 和 LinkedList 的区别？

ArrayList 底层是动态数组，支持随机访问 O(1)，尾部插入均摊 O(1)，中间插入/删除 O(n)，扩容为原来的 1.5 倍。LinkedList 底层是双向链表，不支持随机访问 O(n)，头尾插入/删除 O(1)，但每个节点需要额外存储前后指针。

> 💡 **实战建议**
> 由于 CPU 缓存局部性原理，ArrayList 在绝大多数场景下性能优于 LinkedList，优先使用 ArrayList。

1.3 多线程与并发

Q：线程池的核心参数有哪些？工作流程是怎样的？（高频）

ThreadPoolExecutor 的 7 个核心参数：

参数	含义	说明
corePoolSize	核心线程数	即使空闲也不会被回收（除非设置 allowCoreThreadTimeOut）
maximumPoolSize	最大线程数	核心线程 + 非核心线程的上限
keepAliveTime	空闲存活时间	非核心线程空闲超过此时间会被回收
unit	时间单位	keepAliveTime 的单位
workQueue	任务队列	LinkedBlockingQueue / ArrayBlockingQueue / SynchronousQueue
threadFactory	线程工厂	自定义线程名称、优先级等
handler	拒绝策略	AbortPolicy / CallerRunsPolicy /

参数	含义	说明
		DiscardPolicy / DiscardOldestPolicy

工作流程：提交任务 → 线程数 < corePoolSize？创建核心线程 → 否则放入 workQueue → 队列满且线程数 < maximumPoolSize？创建非核心线程 → 否则执行拒绝策略。

```
// 示例：自定义线程池（阿里规范推荐，不要用 Executors 工厂方法）
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    5,                      // corePoolSize
    10,                     // maximumPoolSize
    60L, TimeUnit.SECONDS,  // keepAliveTime
    new LinkedBlockingQueue<>(100),  // 有界队列，防止 OOM
    new ThreadFactoryBuilder().setNameFormat("biz-pool-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy()  // 调用者线程执行
);
```

> ⚠️ **常见坑**
> Executors.newFixedThreadPool() 使用无界队列 LinkedBlockingQueue，可能导致 OOM；Executors.newCachedThreadPool() 的 maximumPoolSize 为 Integer.MAX_VALUE，可能创建大量线程。阿里开发规范要求手动创建线程池。

Q：synchronized 和 ReentrantLock 的区别？

特性	synchronized	ReentrantLock
实现层面	JVM 关键字（Monitor）	JDK API（AQS）
锁释放	自动释放	需手动 unlock()（finally 中）
可中断	不可中断	lockInterruptibly() 可中断
公平锁	非公平	支持公平/非公平
条件变量	wait/notify（单条件）	Condition（多条件）
锁升级	偏向锁→轻量级锁→重量级锁	无
性能	JDK 1.6 优化后差距不大	差距不大

Q：volatile 关键字的作用和原理？

volatile 有两个作用：**保证可见性**和**禁止指令重排序**。底层通过内存屏障（Memory Barrier）实现。

注意 volatile **不保证原子性**，i++ 这样的复合操作仍然不是线程安全的。

```
// 经典应用：DCL 单例模式（必须加 volatile）
public class Singleton {
    // volatile 防止指令重排：
    // 正常：1.分配内存 → 2.初始化对象 → 3.引用指向内存
    // 重排：1.分配内存 → 3.引用指向内存 → 2.初始化对象
```

```
    // 不加 volatile，线程 B 可能拿到未初始化完成的对象
    private static volatile Singleton instance;


    public static Singleton getInstance() {
        if (instance == null) {              // 第一次检查
            synchronized (Singleton.class) {
                if (instance == null) {      // 第二次检查
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Q：ThreadLocal 的原理？会有什么问题？

每个 Thread 内部维护一个 `ThreadLocalMap`，key 是 ThreadLocal 对象的**弱引用**，value 是存储的值。

内存泄漏问题：GC 后 key（弱引用）变为 null，但 value（强引用）不会被回收。如果线程长期存活（如线程池中的线程），这些 value 就无法被回收。**解决方案：使用完后调用 threadLocal.remove()。**

二、JVM（Java 虚拟机）

2.1 内存区域

Q：JVM 的内存区域是如何划分的？（高频）

区域	线程共享	存储内容	异常
程序计数器	私有	当前线程执行的字节码行号	唯一不会 OOM 的区域
虚拟机栈	私有	栈帧（局部变量表、操作数栈、动态链接）	StackOverflowError / OOM
本地方法栈	私有	Native 方法服务	StackOverflowError / OOM
堆	共享	对象实例和数组	OOM（最常见）
方法区/元空间	共享	类信息、常量、静态变量、JIT 代码	OOM（Metaspace）
直接内存	共享	NIO DirectByteBuffer	OOM

> 💡 **JDK 版本变化**
> JDK 7：字符串常量池从方法区移到堆中。JDK 8：永久代（PermGen）被元空间（Metaspace）取代，使用本地内存而非 JVM 堆内存。

2.2 垃圾回收（GC）

Q：如何判断对象是否可以被回收？

Java 采用**可达性分析法**：从 GC Roots 出发，沿引用链遍历，不可达的对象即为可回收对象。

GC Roots 包括：虚拟机栈中引用的对象、方法区中静态属性引用的对象、方法区中常量引用的对象、本地方法栈中 JNI 引用的对象、被 synchronized 持有的对象。

Q：常见的垃圾收集器有哪些？（高频）

收集器	算法	特点	适用场景
Serial	复制/标记-整理	单线程，STW	客户端模式
ParNew	复制	Serial 多线程版	配合 CMS
Parallel Scavenge	复制	关注吞吐量，JDK 8 默认	后台计算
CMS	标记-清除	低停顿，并发收集，有碎	Web 应用（已废弃）

收集器	算法	特点	适用场景
		片	
G1	分区收集	可预测停顿，JDK 9 默认	大堆、低延迟
ZGC	染色指针+读屏障	停顿 < 1ms，支持 TB 级堆	超低延迟

Q：G1 收集器的原理？

G1 将堆划分为多个大小相等的 Region（1-32MB），每个 Region 可以是 Eden、Survivor、Old 或 Humongous。通过维护优先级列表，**优先回收价值最大的 Region**（Garbage First 名称由来）。

四个阶段：初始标记（STW）→ 并发标记 → 最终标记（STW）→ 筛选回收（STW）。使用 SATB（Snapshot At The Beginning）解决并发标记的漏标问题。可通过 `-XX:MaxGCPauseMillis` 设置期望停顿时间。

Q：对象什么时候会进入老年代？

四种情况：① 对象年龄达到阈值（默认 15，-XX:MaxTenuringThreshold）；② 大对象直接进入老年代（-XX:PretenureSizeThreshold）；③ 动态年龄判断（Survivor 区中相同年龄对象大小之和超过 Survivor 空间一半）；④ Minor GC 后 Survivor 区放不下的对象。

2.3 类加载机制

Q：什么是双亲委派模型？为什么要用它？

类加载器收到加载请求时，先委派给父加载器处理，父加载器无法完成时才自己加载。

```
// 双亲委派的加载器层次
Bootstrap ClassLoader  ← 加载 rt.jar 等核心类库（C++ 实现）
    ↑
Extension ClassLoader  ← 加载 ext 目录
    ↑
Application ClassLoader  ← 加载 classpath（我们写的代码）
    ↑
Custom ClassLoader  ← 自定义类加载器
```

好处：避免类的重复加载；保证核心 API 不被篡改（如自定义 java.lang.String 不会被加载）。

打破双亲委派的场景：SPI 机制（JDBC，使用线程上下文类加载器）、Tomcat 的 WebAppClassLoader（每个 Web 应用独立类加载器）、OSGi 模块化、热部署。

2.4 JVM 调优

Q：常用的 JVM 参数有哪些？

参数	作用	建议值
-Xms / -Xmx	初始/最大堆大小	设为相同值，避免动态扩容
-Xmn	新生代大小	一般为堆的 1/3 ~ 1/2
-XX:MetaspaceSize	元空间初始大小	256m
-XX:+UseG1GC	使用 G1 收集器	JDK 9+ 默认
-XX:MaxGCPauseMillis	G1 期望停顿时间	200（毫秒）
-XX:+HeapDumpOnOutOfMemoryError	OOM 时自动 dump	生产环境必开

三、Spring Boot 框架

3.1 核心原理

Q：Spring Boot 的自动配置原理是什么？（高频）

核心是 `@SpringBootApplication` 注解，它是一个组合注解，包含：

① `@SpringBootConfiguration`（标识配置类）

② `@EnableAutoConfiguration`（开启自动配置，通过 `AutoConfigurationImportSelector` 读取 `META-INF/spring.factories` 中的自动配置类）

③ `@ComponentScan`（组件扫描）

自动配置类通过 `@Conditional` 系列注解进行条件判断，只有满足条件时才生效。用户自定义的 Bean 优先级高于自动配置的 Bean（`@ConditionalOnMissingBean` 保证）。

```
// 自动配置类示例（简化版）
@Configuration
@ConditionalOnClass(DataSource.class)           // classpath 中有 DataSource 类
@ConditionalOnProperty(prefix = "spring.datasource", name = "url")  // 配置了 url
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {


    @Bean
    @ConditionalOnMissingBean  // 用户没有自定义 DataSource 时才生效
    public DataSource dataSource(DataSourceProperties properties) {
        return properties.initializeDataSourceBuilder().build();
    }
}
```

3.2 Spring IoC 与 Bean 生命周期

Q：Spring 中 Bean 的生命周期是怎样的？（高频）

Bean 的完整生命周期可以分为四个阶段：**实例化 → 属性填充 → 初始化 → 销毁**。

```
// Bean 生命周期详细流程
1. 实例化（Instantiation）：通过反射创建 Bean 对象
2. 属性填充（Populate）：依赖注入（@Autowired / @Value）
3. Aware 接口回调：
   - BeanNameAware.setBeanName()
   - BeanFactoryAware.setBeanFactory()
   - ApplicationContextAware.setApplicationContext()
```

```
4. BeanPostProcessor.postProcessBeforeInitialization()
5. InitializingBean.afterPropertiesSet() / @PostConstruct
6. 自定义 init-method
7. BeanPostProcessor.postProcessAfterInitialization()
   ↑ AOP 代理在此步骤创建！
8. Bean 就绪，放入容器
9. DisposableBean.destroy() / @PreDestroy / 自定义 destroy-method
```

Q：Spring 如何解决循环依赖？（高频）

Spring 通过**三级缓存**解决 singleton 作用域的 setter/字段注入的循环依赖：

缓存	名称	存储内容
一级缓存	singletonObjects	完全初始化好的 Bean
二级缓存	earlySingletonObjects	提前暴露的 Bean（已实例化但未完成属性填充）
三级缓存	singletonFactories	Bean 的 ObjectFactory（用于创建早期引用，支持 AOP 代理）

流程（A 依赖 B，B 依赖 A）： 创建 A → A 实例化后将 ObjectFactory 放入三级缓存 → A 填充属性发现依赖 B → 创建 B → B 填充属性发现依赖 A → 从三级缓存获取 A 的 ObjectFactory 创建早期引用放入二级缓存 → B 完成初始化 → A 获取到 B 完成初始化。

> ⚠️ **注意事项**
> 构造器注入的循环依赖无法解决（可用 @Lazy 延迟加载）；prototype 作用域的循环依赖无法解决；Spring Boot 2.6+ 默认禁止循环依赖。

3.3 Spring MVC

Q：Spring MVC 的请求处理流程是怎样的？

```
// Spring MVC 请求处理流程
客户端请求
    ↓
DispatcherServlet（前端控制器，统一入口）
    ↓
HandlerMapping（查找对应的 Controller 方法）
    ↓
HandlerAdapter（适配器，执行 Controller 方法）
    ↓
Controller 方法执行，返回结果
    ↓
┌─ @ResponseBody ──→ HttpMessageConverter 序列化为 JSON → 直接返回
└─ 返回视图名 ──→ ViewResolver 解析视图 → 渲染视图 → 返回 HTML
```

四、微服务架构

4.1 微服务基础

Q：微服务架构中有哪些核心组件？

组件	功能	常用技术
服务注册与发现	服务地址管理	Nacos / Eureka / Consul
配置中心	集中管理配置	Nacos / Apollo / Spring Cloud Config
API 网关	统一入口、路由、鉴权	Spring Cloud Gateway / Zuul
负载均衡	请求分发	Ribbon / Spring Cloud LoadBalancer
服务调用	远程方法调用	OpenFeign / Dubbo
熔断降级	故障隔离与保护	Sentinel / Hystrix / Resilience4j
链路追踪	分布式调用链监控	SkyWalking / Zipkin / Jaeger
消息队列	异步解耦	Kafka / RocketMQ / RabbitMQ

4.2 服务注册与发现

Q：Nacos 和 Eureka 的区别？

特性	Nacos	Eureka
一致性模型	AP + CP（可切换）	仅 AP
服务变更感知	服务端主动推送	客户端定时拉取
配置中心	内置	无
实例类型	临时实例 + 持久实例	仅临时实例
健康检查	客户端心跳 + 服务端探测	客户端心跳
管理控制台	功能丰富	简单
维护状态	活跃维护	2.x 已停止维护

4.3 熔断降级与限流

Q：什么是服务熔断？熔断器的三种状态？

服务熔断是一种保护机制，当某个服务的错误率或响应时间超过阈值时，熔断器打开，后续请求直接

返回失败（快速失败），防止故障扩散（**雪崩效应**）。

```
// 熔断器状态转换
CLOSED（关闭，正常调用）
  │ 错误率/慢调用比例超过阈值
  ↓
OPEN（打开，直接失败）
  │ 等待一段时间（熔断时长）
  ↓
HALF-OPEN（半开，允许少量探测请求）
  │ 探测成功 → 回到 CLOSED
  │ 探测失败 → 回到 OPEN
```

4.4 分布式相关

Q：CAP 理论和 BASE 理论是什么？

CAP 理论：分布式系统不可能同时满足一致性（C）、可用性（A）、分区容错性（P），最多满足其中两个。由于网络分区不可避免，实际上是在 CP 和 AP 之间选择。

BASE 理论：Basically Available（基本可用）、Soft state（软状态）、Eventually consistent（最终一致性）。是对 CAP 中 AP 的延伸，大规模互联网系统的实践指导。

Q：分布式锁的实现方案有哪些？

方案	实现方式	优点	缺点
Redis	SET key NX EX + Lua 释放	性能高	主从切换可能丢锁
Redisson	看门狗续期 + RedLock	功能完善、可重入	RedLock 有争议
Zookeeper	临时顺序节点 + Watch	可靠性高、强一致	性能不如 Redis
数据库	SELECT FOR UPDATE / 版本号	实现简单	性能差，不推荐高并发

```
// Redis 分布式锁示例（Redisson）
RLock lock = redissonClient.getLock("order:lock:" + orderId);
try {
    // 尝试加锁，最多等待 3 秒，锁自动过期 10 秒
    // Redisson 的看门狗会自动续期（默认 30 秒续一次）
    if (lock.tryLock(3, 10, TimeUnit.SECONDS)) {
        // 执行业务逻辑
        doBusinessLogic();
    }
} finally {
    if (lock.isHeldByCurrentThread()) {
        lock.unlock();
```

```
    }
}
```

五、AOP（面向切面编程）

5.1 AOP 基础

Q：Spring AOP 中有哪些通知类型？执行顺序是怎样的？

通知类型	注解	执行时机
前置通知	@Before	方法执行前
后置通知	@After	方法执行后（无论是否异常，类似 finally）
返回通知	@AfterReturning	方法正常返回后
异常通知	@AfterThrowing	方法抛出异常后
环绕通知	@Around	包裹目标方法，可控制是否执行及修改返回值

正常执行顺序（Spring 5.2.7+）： Around（前）→ Before → 目标方法 → AfterReturning → After → Around（后）

异常执行顺序： Around（前）→ Before → 目标方法抛异常 → AfterThrowing → After

5.2 AOP 实现原理

Q：JDK 动态代理和 CGLIB 代理的区别？（高频）

特性	JDK 动态代理	CGLIB 代理
基于	接口	继承（生成子类）
实现	Proxy + InvocationHandler	ASM 字节码 + MethodInterceptor
要求	目标类必须实现接口	目标类不能是 final
性能	反射调用，JDK 高版本已优化	字节码直接调用，略快
Spring Boot 默认	2.x 之前（有接口时）	2.x 起默认使用 CGLIB

Q：为什么同一个类中方法调用 AOP 不生效？如何解决？（高频）

因为 Spring AOP 基于代理实现，同一个类中 `this.method()` 调用的是目标对象本身而非代理对象，所以 AOP 不生效。

```
// 问题示例
```

```
@Service
public class OrderService {
    public void createOrder() {
        // this 指向的是原始对象，不是代理对象，AOP 不生效！
        this.sendNotification();
    }

    @OperationLog("发送通知")
    public void sendNotification() { ... }
}

// 解决方案一：注入自身
@Service
public class OrderService {
    @Autowired
    private OrderService self;  // 注入的是代理对象

    public void createOrder() {
        self.sendNotification();  // 通过代理对象调用，AOP 生效
    }
}

// 解决方案二：AopContext（需要 exposeProxy = true）
((OrderService) AopContext.currentProxy()).sendNotification();
```

5.3 自定义 AOP 注解实战

Q：如何自定义一个 AOP 注解？（附完整示例）

以下是一个完整的「接口耗时日志」注解实现，包含注解定义、切面逻辑和使用方式：

```
// 第一步：定义注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface ApiLog {
    String value() default "";  // 接口描述
}

// 第二步：编写切面
@Aspect
@Component
@Slf4j
public class ApiLogAspect {

    @Around("@annotation(apiLog)")
    public Object around(ProceedingJoinPoint joinPoint, ApiLog apiLog) throws Throwable {
        String methodName = joinPoint.getSignature().toShortString();
        String desc = apiLog.value();
        Object[] args = joinPoint.getArgs();
```

```
        log.info("[{}] 开始执行, 参数: {}", desc, JSON.toJSONString(args));
        long start = System.currentTimeMillis();

        try {
            Object result = joinPoint.proceed();  // 执行目标方法
            long cost = System.currentTimeMillis() - start;
            log.info("[{}] 执行成功, 耗时: {}ms, 返回: {}", desc, cost, result);
            return result;
        } catch (Throwable e) {
            long cost = System.currentTimeMillis() - start;
            log.error("[{}] 执行异常, 耗时: {}ms, 异常: {}", desc, cost, e.getMessage());
            throw e;
        }
    }
}

// 第三步: 使用注解
@RestController
public class UserController {

    @ApiLog("查询用户信息")
    @GetMapping("/user/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getById(id);
    }
}

// 输出效果:
// [查询用户信息] 开始执行, 参数: [1001]
// [查询用户信息] 执行成功, 耗时: 23ms, 返回: User(id=1001, name=张三)
```

六、事务

6.1 事务基础

Q：事务的 ACID 特性是什么？

特性	含义	实现机制（InnoDB）
原子性 Atomicity	操作要么全部完成要么全部不完成	undo log（回滚日志）
一致性 Consistency	事务前后数据库保持一致状态	是目的，由 A/I/D 共同保证
隔离性 Isolation	并发事务之间互不干扰	锁 + MVCC
持久性 Durability	事务提交后数据永久保存	redo log（重做日志）

Q：事务的隔离级别有哪些？

隔离级别	脏读	不可重复读	幻读	说明
Read Uncommitted	可能	可能	可能	最低级别
Read Committed	解决	可能	可能	Oracle 默认
Repeatable Read	解决	解决	InnoDB 基本解决	MySQL 默认
Serializable	解决	解决	解决	完全串行，性能最差

> 💡 **MySQL 特殊之处**
> InnoDB 在 RR 级别下通过 MVCC + Next-Key Lock 在很大程度上解决了幻读问题，这是 MySQL 的特殊优化，其他数据库的 RR 级别通常无法解决幻读。

Q：InnoDB 的 MVCC 是如何实现的？（高频）

MVCC 通过为每行数据维护多个版本来实现非锁定读。每行数据有两个隐藏列：`DB_TRX_ID`（最近修改该行的事务 ID）和 `DB_ROLL_PTR`（指向 undo log 中该行的上一个版本）。

核心机制是 **ReadView（读视图）**：在 RC 级别下每次 SELECT 都创建新的 ReadView，在 RR 级别下只在事务第一次 SELECT 时创建。通过比较行的 DB_TRX_ID 与 ReadView 中的活跃事务列表，判断该版本是否对当前事务可见，不可见则沿 undo log 版本链向前查找。

6.2 Spring 事务管理

Q：Spring 事务的传播行为有哪些？（高频）

传播行为	含义	典型场景
REQUIRED（默认）	有事务则加入，没有则新建	大多数业务方法
REQUIRES_NEW	总是新建事务，挂起当前事务	操作日志（业务回滚日志不回滚）
NESTED	嵌套事务（Savepoint），可独立回滚	批量处理中单条失败不影响整体
SUPPORTS	有事务则加入，没有则非事务执行	查询方法
NOT_SUPPORTED	非事务执行，挂起当前事务	不需要事务的操作
MANDATORY	必须在事务中，否则抛异常	强制要求调用方有事务
NEVER	必须非事务，否则抛异常	不允许在事务中执行的操作

REQUIRED vs REQUIRES_NEW vs NESTED 的核心区别：

```
// REQUIRED：内外同一个事务，任何一方回滚 → 整个事务回滚
@Transactional
public void methodA() {
    methodB();  // B 加入 A 的事务
    // B 抛异常 → A 也回滚
}

// REQUIRES_NEW：内外独立事务
@Transactional
public void methodA() {
    methodB();  // B 创建新事务，A 的事务被挂起
    // B 回滚不影响 A，A 回滚也不影响已提交的 B
}

// NESTED：嵌套事务（Savepoint）
@Transactional
public void methodA() {
    methodB();  // B 在 A 的事务中创建 Savepoint
    // B 回滚只回滚到 Savepoint，不影响 A
    // 但 A 回滚会导致 B 也回滚
}
```

6.3 事务失效场景（高频必问）

Q：@Transactional 在哪些情况下会失效？

这是面试中的超高频问题，以下是 8 种常见的事务失效场景：

失效场景	原因	解决方案
方法非 public	Spring 事务拦截器检查修饰符	改为 public 方法

失效场景	原因	解决方案
自调用（this.method()）	绕过了代理对象	注入自身 / AopContext
异常被 catch 吞掉	Spring 感知不到异常	catch 后重新 throw
异常类型不匹配	默认只回滚 RuntimeException	rollbackFor = Exception.class
数据库引擎不支持	MyISAM 不支持事务	使用 InnoDB
Bean 未被 Spring 管理	没有 @Service 等注解	确保被容器管理
传播行为设置不当	NOT_SUPPORTED / NEVER	检查传播行为配置
多线程调用	事务基于 ThreadLocal	同一线程内操作

```
// 最常见的失效场景示例


// ❌ 场景 1：异常被吞掉
@Transactional
public void transfer() {
    try {
        deduct();   // 扣款
        addMoney(); // 加款（抛异常）
    } catch (Exception e) {
        log.error("转账失败", e);
        // 异常被 catch 了，Spring 感知不到，不会回滚！
    }
}


// ✅ 正确做法：catch 后重新抛出
@Transactional(rollbackFor = Exception.class)
public void transfer() {
    try {
        deduct();
        addMoney();
    } catch (Exception e) {
        log.error("转账失败", e);
        throw e;  // 重新抛出，让 Spring 感知到异常
    }
}


// ❌ 场景 2：异常类型不匹配
@Transactional  // 默认只回滚 RuntimeException
public void importData() throws IOException {
    // IOException 是 checked exception，不会触发回滚！
    saveToDb();
    writeToFile();  // 抛出 IOException
}


// ✅ 正确做法：指定 rollbackFor
@Transactional(rollbackFor = Exception.class)
public void importData() throws IOException { ... }
```

6.4 分布式事务

Q：分布式事务有哪些解决方案？如何选型？

方案	原理	一致性	侵入性	适用场景
2PC / XA	协调者两阶段提交	强一致	低	参与者少、强一致要求
TCC	Try-Confirm-Cancel	最终一致	高	高性能、资金类业务
SAGA	正向操作 + 补偿操作	最终一致	中	长流程业务
本地消息表	本地事务 + 消息表 + 定时扫描	最终一致	中	简单场景
可靠消息	RocketMQ 事务消息	最终一致	低	异步解耦场景
Seata AT	自动补偿（undo log）	最终一致	低	对业务无侵入

💡 **选型建议**

大多数互联网场景选择最终一致性方案（可靠消息或 Seata AT），因为强一致性方案的性能开销太大。资金类业务推荐 TCC，长流程推荐 SAGA。

七、补充高频题

7.1 设计模式在 Spring 中的应用

Q：Spring 中用到了哪些设计模式？

设计模式	应用场景	具体实现
工厂模式	Bean 创建	BeanFactory / FactoryBean
单例模式	Bean 默认作用域	DefaultSingletonBeanRegistry
代理模式	AOP	JDK 动态代理 / CGLIB
模板方法	消除样板代码	JdbcTemplate / RestTemplate
观察者模式	事件机制	ApplicationEvent / ApplicationListener
适配器模式	Handler 适配	HandlerAdapter
策略模式	资源加载	Resource 接口的不同实现
责任链模式	请求处理链	Filter Chain / Interceptor Chain

7.2 MySQL 锁机制

Q：InnoDB 中有哪些锁？

锁类型	说明	作用
Record Lock	锁定单行记录	精确锁定
Gap Lock	锁定索引记录之间的间隙	防止幻读（RR 级别）
Next-Key Lock	Record Lock + Gap Lock	InnoDB 默认行锁算法
共享锁（S）	读锁，SELECT ... LOCK IN SHARE MODE	允许其他事务读
排他锁（X）	写锁，SELECT ... FOR UPDATE	阻塞其他事务读写
意向锁（IS/IX）	表级锁	快速判断表中是否有行锁

使用建议

1. 理解优于背诵：结合实际项目经验理解每个知识点，面试时能说出自己的理解和实践经验，比单纯背诵答案更有说服力。

2. 重点关注高频题：标注了「高频」的题目是面试中出现概率最高的，务必重点准备。包括 HashMap 原理、线程池参数、Spring 自动配置、Bean 生命周期、循环依赖、事务失效场景等。

3. 代码示例要能手写：面试中可能要求手写代码，如 DCL 单例、自定义线程池、自定义 AOP 注解等，建议多练习。

4. 深度追问要有准备：面试官通常会追问「为什么」和「怎么实现的」，如 HashMap 为什么用红黑树、synchronized 锁升级过程、MVCC 的 ReadView 判断规则等，要准备到源码级别。

5. 结合场景回答：回答分布式事务、微服务治理等问题时，结合自己项目中的实际选型和踩坑经验，会大大加分。